

01 — MEMORY, OBJECT LIFETIME & RAI

The single most-tested C++ area. Interviewers probe whether you understand *when* objects are created/destroyed, *where* they live, and how ownership is expressed in the type system.

Take-home lesson. In C++ you don't manage memory — you manage *lifetimes*. Every resource bug is a lifetime bug in disguise: used too early, freed too late, or owned by no one.

1. STORAGE DURATIONS (KNOW ALL FOUR COLD)

Duration	Where	Lifetime	Examples
automatic	stack	enclosing scope	local non-static vars, params
static	data segment	whole program	globals, <code>static</code> locals, <code>static</code> members
dynamic	heap (free store)	until you <code>delete</code> /reset	<code>new</code> , <code>make_unique</code> , <code>make_shared</code>
thread	per-thread storage	thread lifetime	<code>thread_local</code>

Gotchas:

- Static local init is lazy and thread-safe since C++11 ("magic statics" / Meyers singleton). The compiler inserts a guard variable + atomic check. Pre-C++11 this was a data race.
- Static init order fiasco (SIOF): order of init of non-local statics across *different* translation units is unspecified. Fix with the *construct-on-first-use* idiom (function-local static returned by reference), or `constexpr` (C++20) for compile-time-initialized statics.

- Zero-init happens before dynamic init for statics.

```
Logger& log() { // construct-on-first-use, dodges SIOF
    static Logger instance; // thread-safe since C++11
    return instance;
}
```

War story. A team shipped a global config object that read another global logger in its constructor. Worked on the dev box, crashed at customer startup: different TU link order → logger not yet constructed. The fix wasn't more `mutex` — it was turning both into construct-on-first-use functions so the *dependency* forced the order. The bug was never about threads; it was about initialization order.

Stop and think. Two globals in different `.cpp` files, each using the other in its constructor. Is there an ordering that always works?

► answer

2. OBJECT LIFETIME: THE EXACT RULES

An object's lifetime begins when storage is obtained *and* initialization completes; it ends when the destructor call starts (or storage is released for trivial types).

Key facts interviewers dig into:

- Using an object before its lifetime begins or after it ends = UB, even if the memory is still there. This is why type-punning through a pointer is UB unless the object was created there (C++23 adds `std::start_lifetime_as`).
- Temporaries live until the end of the *full-expression* — except when bound to a `const&` or `&&` reference, which extends lifetime to the reference's scope. Lifetime extension does not chain through function returns or through a member reference.
- Destruction order = reverse of construction. For members: reverse of *declaration* order (not initializer-list order — a classic warning source).

```
const std::string& r = std::string("hi"); // lifetime extended to r's
scope — OK
const std::string& bad = getVec()[0];      // NOT extended — dangling if
temp vector
```

C++17+ traps worth showing:

```
// Structured bindings + lifetime extension DO cooperate:
const auto& [a, b] = std::pair{1, std::string("x")}; // temp pair
extended — OK

// Range-based for evaluates its range expression ONCE and binds it.
// Pre-C++23 this bites when the range is a temporary sub-object:
for (char c : getObj().name()) { ... } // getObj() temp extended...
for (char c : getVec()[0].chars()) { ... } // ...but a temp's SUBOBJECT is
NOT (dangling)
// C++23 fixed the range-for temporary lifetime rules for exactly this.
```

Take-home lesson. Lifetime extension is a *single-hop* favor: it saves the exact temporary you bind, never something reachable *through* it.

Stop and think. `std::string_view sv = std::string("hello");` — legal? Safe?

► answer

3. RAI — THE ORGANIZING PRINCIPLE

Resource Acquisition Is Initialization: bind a resource's lifetime to an object's lifetime. Acquire in the constructor, release in the destructor. The destructor runs deterministically on scope exit — including during stack unwinding from an exception. This is why C++ needs no `finally`.

Why it wins:

- Exception safety for free — resources release on any exit path.
- Composable: members clean themselves up.
- Makes ownership explicit in the type.

The Rule of Zero: if your class holds resources only through RAll members (smart pointers, containers), write *no* special members — let the compiler generate them. Only write destructors/copy/move when you manage a raw resource directly.

A generic scope-guard (handy, and a nice thing to be able to write on a board):

```
template <class F>
struct ScopeGuard {
    F f; bool active = true;
    ~ScopeGuard() { if (active) f(); }
    void dismiss() { active = false; }
};
template <class F> ScopeGuard(F) -> ScopeGuard<F>;    // C++17 CTAD guide

auto g = ScopeGuard{[&]{ close(fd); } };              // runs on any exit
path
// ... if we succeed and want to keep fd open: g.dismiss();
```

Take-home lesson. Anywhere you'd write `try { ... } finally { cleanup(); }` in another language, in C++ you write a destructor once and never think about the cleanup path again.

4. SMART POINTERS (OWNERSHIP IN THE TYPE SYSTEM)

`unique_ptr` — sole ownership, zero overhead

- Move-only (copy deleted). `sizeof` == raw pointer (with stateless deleter).
- Prefer `std::make_unique<T>(args)` — exception-safe, no `new` in your code.
- Custom deleter is part of the type: `unique_ptr<FILE, decltype(&fclose)>`.
A stateless deleter (empty class via EBO) keeps size == 1 pointer; a function-pointer deleter grows the object.
- Works for arrays: `unique_ptr<T[]>`.

```
// RAII over a C API, C++17:
auto file = std::unique_ptr<FILE, decltype(&std::fclose)>(
    std::fopen("data.bin", "rb"), &std::fclose);
if (!file) return; // fclose runs automatically on scope
exit
```

`shared_ptr` — shared ownership, ref-counted

- Two counts in the control block: strong (owners) + weak. Object destroyed at `strong==0`; control block freed at `strong==0 && weak==0`.
- `sizeof(shared_ptr)` == 2 pointers (object ptr + control-block ptr).
- Ref-count ops are atomic (thread-safe count, NOT thread-safe pointee).
- `make_shared` VS `shared_ptr(new T)`: `make_shared` does one allocation (object + control block together) — faster, better locality, exception-safe. Downside: the object's memory isn't freed until the *last weak_ptr* dies (they share the block). With big objects + long-lived weak_ptrs, prefer separate allocation.

`weak_ptr` — non-owning observer

- Breaks reference cycles (the classic `shared_ptr` leak). Parent → child `shared_ptr`, child → parent `weak_ptr`.
- `lock()` returns a `shared_ptr` (nullptr if expired) — the only safe way to access. `expired()` alone is racy.

```
struct Node {
    std::shared_ptr<Node> next; // owns forward
    std::weak_ptr<Node> prev; // observes back — no cycle
};
// If prev were shared_ptr, a two-node ring would leak: each keeps the
// other at
// refcount 1 forever. weak_ptr sees the node without extending its life.
```

`enable_shared_from_this`

- Lets a member get a `shared_ptr` to `this` that shares the *existing* control block. Creating `shared_ptr(this)` directly = double-free.

Interview one-liners:

- "Default to `unique_ptr`; reach for `shared_ptr` only when ownership is genuinely shared." Shared ownership is often a design smell.
- Passing conventions: sink → `unique_ptr` by value; "I need to keep a share" → `shared_ptr` by value; just using it → pass `T*` or `T&`.

Take-home lesson. The smart pointer *type* is documentation the compiler enforces: `unique_ptr` says "I alone own this," `shared_ptr` says "we share," `weak_ptr` says "I'm just watching," raw `T*` / `T&` says "I don't own this."

Stop and think. Why is `foo(std::shared_ptr<T>(new T), bar())` historically unsafe?

► answer

5. `NEW` / `DELETE` MECHANICS (STILL ASKED)

- `new T` = `operator new` (allocate) + constructor. `delete` = destructor + `operator delete`.
- Mismatched forms (`new[]` with `delete`) = UB.
- Placement new: `new (ptr) T(args)` constructs into existing storage; you must call `~T()` manually. Basis of allocators, small-buffer optimization, `std::vector`'s uninitialized capacity.
- Overriding `operator new` / `delete` (class-level or global) for pools/tracking.
- `new` throws `std::bad_alloc`; `new(std::nothrow)` returns null.

```
alignas(T) unsigned char buf[sizeof(T)];
T* p = new (buf) T{args};    // placement new — no heap allocation
p->~T();                     // you own the destruction
```

6. COMMON UB / LEAKS TO NAME IN INTERVIEWS

- Dangling pointer/reference (return address of local; ref/ `string_view` to

expired temp).

- Double-free / use-after-free.
- Buffer overrun, off-by-one.
- Reading uninitialized memory.
- Leaking on the exception path (why raw `new` between operations is unsafe).
- Slicing (copying a derived into a base value — loses the derived part).
- Iterator/reference invalidation after container mutation.

Take-home lesson. If you can name the failure mode, you can design it out.
Almost every item on this list disappears under Rule-of-Zero RAI + smart pointers + values instead of owning raw pointers.

DRILL PROMPTS

1. Walk the exact sequence when `auto p = make_shared<Foo>()` throws in Foo's ctor.
2. When is `make_shared` the *wrong* choice?
3. Why is member destruction order tied to declaration order, and when does it bite?
4. Implement a minimal `unique_ptr` (move ctor/assign, `release`, `reset`, `get`).
5. Show a reference-cycle leak and fix it with `weak_ptr`.
6. Explain why `string_view = std::string("x")` dangles but `const string& = std::string("x")` doesn't.
7. Write the `ScopeGuard` above from scratch, with the CTAD deduction guide.